A1-1. Field Verification System Design Document [Sender]

October 10, 2026 — Prepared by Attractor Co., Ltd.

> This document is the Milestone 1 deliverable "A1-1. Field Verification System Design Document [Sender]" and satisfies acceptance criterion A2 (it includes a system diagram, hardware definitions, and software function descriptions).

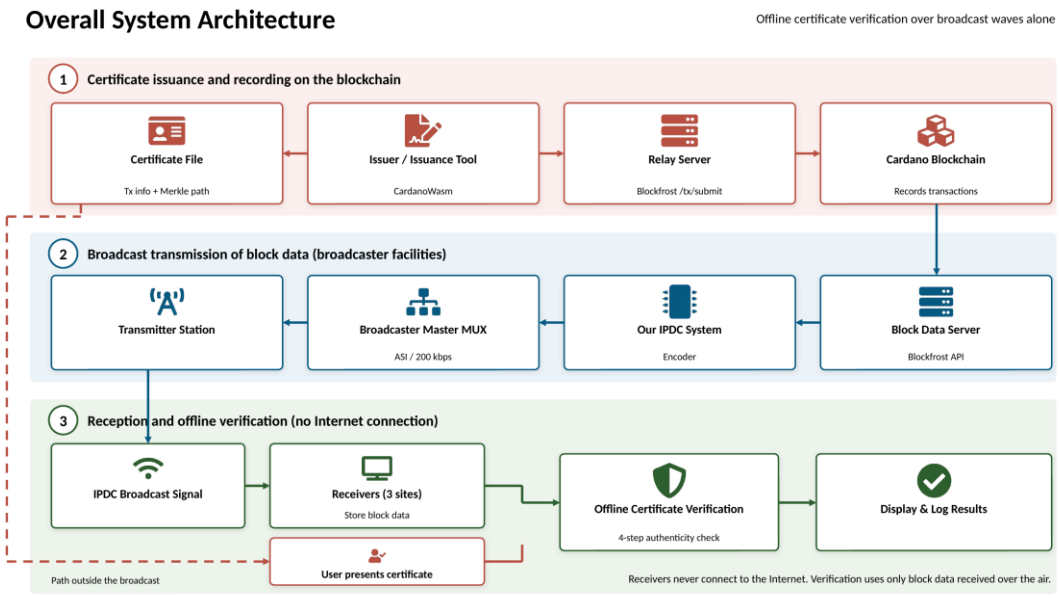
Overview and Positioning

This document corresponds to the Milestone 1 deliverable "A1-1. Field Verification System Design Document [Sender]" and defines the sender-side system (transmission from the broadcaster).

The scope covers everything from retrieving block header information from the Cardano blockchain to transmitting it on broadcast waves via IPDC. The receiver-side design is defined separately in A1-2.

Overall System Architecture (Background)

As background for this document, the overall system architecture of the project is shown below. The system consists of three domains: certificate issuance, broadcast transmission, and reception and verification. This document (A1-1) covers the broadcast transmission domain; the reception and verification domain is defined in A1-2. The certificate issuance domain reuses the implementation prototyped in the previous project.



Overall system architecture

In the certificate issuance domain, the issuing organization generates a certificate with the certificate issuance tool (which builds a transaction with CardanoWasm and stores a JSON document in a verifiable-credential format in the transaction) and records it on the Cardano blockchain via a relay server. The user receives a certificate file that contains the transaction information and the Merkle path.

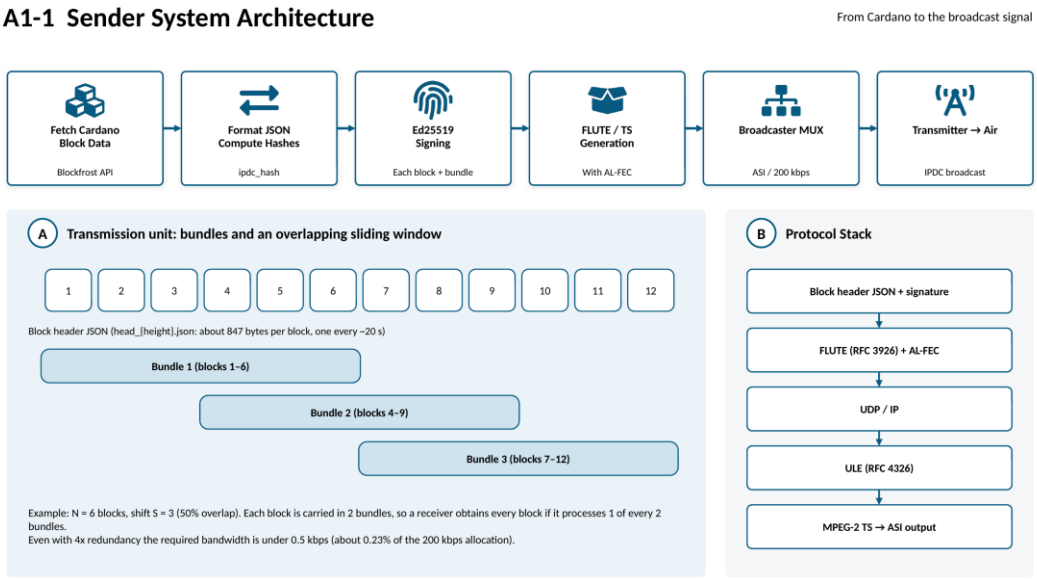
In the broadcast transmission domain, the block data server retrieves block header information from Cardano and formats it as a custom JSON document. Our IPDC system converts this into a broadcast signal and feeds it into the broadcaster's master-control MUX. The following sections of this document define this domain in detail.

In the reception and verification domain, the receiver obtains block information from the broadcast waves, stores it locally, and verifies the

authenticity of a presented certificate offline by matching the two. Details are defined in A1-2.

The key point of this architecture is that the receiver can verify certificates using only the block information received over the air, without ever connecting to the Internet. There is no communication path between the certificate issuance domain and the reception and verification domain other than the broadcast waves. The role of the sender side is to deliver, over the broadcast waves alone, all the information required for verification, complete and in a tamper-proof form.

System Diagram



Sender system architecture

The sender-side roles are the blocks shown in the diagram above. Discussions with the broadcaster have already been completed. The output of the IPDC system we provide (ASI, MPEG-2 TS compliant) is fed into one

spare port (allocated bandwidth: 200 kbps) of the MUX in the master-control system currently operated by the broadcaster, and is multiplexed with the regular broadcast signal for transmission.

Hardware Definitions

Component	Expected Equipment / Specifications	Role
Block data server	Existing implementation. A server that retrieves Cardano block information through an external API such as Blockfrost (source code reused from the previous project)	Retrieves block header information from the Cardano blockchain
Data conversion module (implemented on the block data server)	No dedicated hardware; implemented as a software function on the block data server	Converts block information into the IPDC transmission format
Our IPDC system (encoder)	IPDC encoder developed in-house (ASI output, MPEG-2 TS compliant, allocated bandwidth 200 kbps). Development is complete up to generating IPDC from block information and outputting it as TS over ASI	Converts and encodes block information into an IPDC broadcast signal and outputs it to the MUX
Broadcaster master-control MUX (existing)	Transmission multiplexer currently operated by the broadcaster (ASI input, one spare port, allocated bandwidth 200 kbps)	Multiplexes our IPDC output with the regular broadcast signal for transmission
Transmission facility (transmitter station)	The broadcaster's actual transmission facility (cooperating station and transmitter site to be confirmed)	Transmits the IPDC-multiplexed signal as actual TV broadcast waves
Monitoring and logging equipment	Network analyzer / transmission log collection PC	Records transmitted content and timing and collects logs for comparison during testing

Data Design (Transmitted Data Structure)

This data design follows the prototype built in a previous project (validated first on the Symbol blockchain and then adapted for Cardano).

What is transmitted over IPDC is block header information retrieved from the Cardano blockchain and converted into a custom JSON format, with the file name head_{block height}.json. A block header JSON is generated each time Cardano produces a block (approximately every 20 seconds), but transmission is performed in bundles, as described below. The measured size is approximately 847 bytes per block (830–910 bytes).

Field Name	Content / Source
block_vrf, confirmations, epoch, epoch_slot, fees, hash, height, next_block, op_cert, op_cert_counter, output, previous_block, size, slot, slot_leader, time, tx_count	Retrieved from the Blockfrost API (standard Cardano block information)
block_body_hash	Transaction root. The root of a custom SHA3-256 Merkle tree (leaf = SHA3-256(transaction ID as a hexadecimal string); internal node = SHA3-256(left hex string + right hex string); when the number of elements is odd, the last element is duplicated). For a block with no transactions, SHA3-256(empty string). This value is matched against the certificate's Merkle path during verification
ipdc_hash	SHA3-256(height (64-bit integer, 8 bytes, little-endian) ‖ hash ‖ previous_block ‖ ipdc_previous_block ‖ block_body_hash). Reproduced and verified against 18 blocks of real data
ipdc_previous_block	ipdc_hash of the preceding block

Because ipdc_previous_block is part of the input to ipdc_hash, the link to the preceding block is cryptographically bound. This allows block headers received over the air to be chained together independently, and the receiver can verify consistency by tracing back to a known, trusted block header (the trust anchor). If ipdc_previous_block is altered in any intermediate block, that block's ipdc_hash changes and every subsequent link breaks.

All hash functions used in this design are SHA3-256. Note that block_body_hash is not Cardano's native block body hash (BLAKE2b-256); it is a Merkle root that this system constructs from the list of transaction IDs retrieved from Blockfrost. For verification to succeed, the Merkle path on the certificate side must therefore be generated with the same tree construction rules (SHA3-256; leaves are transaction IDs as hexadecimal strings; internal nodes hash the concatenation of the left and right hexadecimal strings). To allow third parties to perform verification independently, this design assumes that the tree construction rules and the input order and endianness of ipdc_hash are published as a specification.

Transmission protocol

For IPDC transmission, the JSON data above is split for file delivery with FLUTE (RFC 3926), packetized into UDP/IP, encapsulated with ULE (RFC 4326), and sent to the MUX in MPEG-2 TS format.

Transmission Unit and Signing Unit

The transmission unit (FLUTE object) and the signing unit are separated. Each FLUTE object is a "bundle" that groups multiple blocks. Each block

header JSON in a bundle is signed individually, and a manifest signature covering the entire bundle is added on top.

In an earlier experiment, one object was transmitted each time a block was produced (approximately every 20 seconds), and the receiver's recovery processing could not keep up with the transmission cycle, causing objects to be missed. Grouping multiple blocks into one object gave the receiver more processing time and improved the situation, but did not fully resolve it. This shows that the fixed per-object processing cost (session handling, FDT parsing, FEC decoder initialization, file writing, and parsing) dominates, so what must be reduced is the number of objects per unit time, not the number of bytes transmitted.

Individual signatures are kept within each bundle so that a block obtained from an adjacent bundle through overlapping transmission can be verified on its own, even if the manifest of its original bundle was not received. Individual signatures cost 64 bytes per block, only a few percent of the whole bundle.

Overlapping Sliding Window

Bundles are transmitted with overlap. If the bundle size is N blocks and the shift to the next bundle is S blocks, each block is contained in N/S bundles.

The effect of this method differs from simple repeated transmission (carousel). Periods when the receiver's processing is congested are contiguous, so if the same bundle is repeated back to back, all copies are likely to be missed together. By shifting the bundles, however, the receiver can obtain every block as long as it processes one out of every N/S bundles.

In other words, the hard real-time requirement of "process every object within its deadline" is relaxed to "be on time once in every N/S attempts."

The estimates below are based on real data (847 bytes per block, equivalent to 288 bytes after zstd compression including the signature). Against the allocated bandwidth of 200 kbps, even fourfold redundancy uses only 0.23%.

N	S	Overlap	Transmission Cycle	Redundancy	Object Size	Required Bandwidth	Effective Processing Margin
30	15	50%	5 min	2x	8.4 KB	231 bps	10 min
30	10	67%	3 min 20 s	3x	8.4 KB	346 bps	10 min
36	9	75%	3 min	4x	10.1 KB	461 bps	12 min
60	30	50%	10 min	2x	16.9 KB	231 bps	20 min
90	30	67%	10 min	3x	25.3 KB	346 bps	30 min

N and S will be finalized based on the results of Test 7 in B1 (determining the bundle size and overlap width). Compression matters less for saving bandwidth than for lowering recovery processing time by reducing object size (about 28 KB uncompressed and about 8.4 KB after zstd compression at N = 30). The trade-off against decompression time will also be measured in Test 7.

Note that overlapping transmission and AL-FEC protect against different things. AL-FEC is effective against packet loss within an object (caused by radio propagation) but does nothing when the receiver misses an entire object for processing reasons. Overlapping transmission is the opposite. The two are used together.

Manifest Structure

Each bundle includes a manifest.

Item	Content
Range	Start and end heights of the blocks in the bundle
Block list	height and ipdc_hash of each block
End-of-range information	hash and number of confirmations of the latest block at the end of the range
Issue time	Time the manifest was generated
Revoked key ID list	Identifiers of revoked signing keys. Serves as the delivery path for revocation information to offline receivers
Next key announcement	Signing key scheduled for rollover, signed with the current key or a higher-level key
Key ID and signature	Ed25519 signature over the entire manifest

The manifest lets the receiver identify the blocks it has missed as authenticated information. This is important for preventing unnoticed gaps, and it provides the basis for distinguishing a verification result of "rejected" from "undetermined because not received." Also, because the confirmations field in the block header JSON is the value at retrieval time and does not indicate finality, the number of confirmations at the end of the range is carried in the manifest.

Immutability of Transmitted Data

Once a block header JSON has been transmitted, it is never changed. In overlapping transmission the same block is included in multiple bundles, so if the JSON were regenerated for every transmission its content would change, and the receiver would see multiple blocks with different content and different signatures for the same height.

Specifically, among the retrieved block information, confirmations and next_block change depending on when they are retrieved (a mix of values has been confirmed in real data). The JSON from the first retrieval is either frozen and reused for all later retransmissions, or these mutable fields are excluded from transmission. Since they carry no information the receiver can verify, excluding them causes no harm.

If a rollback occurs, already-transmitted files are not rewritten; the new data is recorded as a separate series, and the reconstruction is stated explicitly in the manifest.

Security Design (Tamper Prevention and Anti-Spoofing)

Assumptions and Trust Boundary

What this system transmits is public block header information from the Cardano blockchain, and there is no problem if third parties receive or view it. The properties this design must protect are therefore not confidentiality but authenticity and integrity.

Broadcast waves can be received by anyone, but extracting IPDC data requires a dedicated reception and demodulation environment and is technically demanding. However, this is a practical barrier to entry and is not treated as a basis for security (security by obscurity).

The transmission path (our IPDC system → broadcaster master-control MUX → transmitter station → antenna) is a controlled environment within the broadcaster's licensed facilities, and the assumption holds that the broadcaster's operations will not tamper with the transmitted data. This design treats this path as inside the trust boundary.

Assumed Threats and Countermeasures

Threat	Description	Countermeasure in This Design
Tampering within the transmission path	Transmitted data is rewritten inside the broadcaster's facilities	These are licensed facilities under the broadcaster's control, so the operational risk is low. In addition, tampering can be detected by signature verification on the receiver side
Transmission errors and losses	Bit errors and packet losses during radio propagation or multiplexing	Detected by FLUTE FEC and per-JSON signature verification; recovered through repeated carousel transmission
Spoofed transmission	A third party transmits fake IPDC data from a different transmitter so that receivers ingest fake block headers	Digital signatures on transmitted data and pre-registration of the verification public key in receivers
History tampering	Past block headers are replaced to make a fake certificate appear legitimate	Verification of the hash chain formed by ipdc_hash / ipdc_previous_block back to the trust anchor
Replay	Past legitimate data is retransmitted to make an old state appear current	Monotonic-increase checks on height, slot, and time, and detection of deviation from the receiver's internal clock

Digital Signatures on Transmitted Data

The hash chain formed by ipdc_hash / ipdc_previous_block prevents the history of previously received block headers from being rewritten later. However, because ipdc_hash can be computed from public information alone, it is possible in principle for a third party to create a new fake block header that correctly links to the latest legitimate block. The receiver therefore needs a separate means of determining whether received data was sent by the legitimate source.

In this design, the sender attaches a digital signature (Ed25519 assumed) to each block header JSON (or to a manifest covering a fixed range) and transmits the signature value together with the key ID. The corresponding verification public key is pre-registered in each receiver at installation. An Ed25519 signature is 64 bytes, about 70 bytes including the key ID, which is a few percent of the block header JSON body (about 1–1.5 KB) and has a negligible effect on the 200 kbps allocated bandwidth.

Selection of the Signature Scheme

The signature algorithm adopted is **Ed25519 (EdDSA, RFC 8032)**. A comparison with the main candidates is shown below.

Scheme	Signature Size	Public Key Size	Implementation Risk	CRYPTREC	Cardano Alignment
Ed25519 (adopted)	64 B	32 B	Low (deterministic; no random number needed when signing)	e-Government Recommended Ciphers List	Native
ECDSA P-256	64 B (DER 70–72 B)	33 B	Medium to high (high-quality random number required for every signature)	e-Government Recommended Ciphers List	Limited
RSA-PSS 2048	256 B	approx. 294 B	Low to medium	e-Government Recommended Ciphers List	No
Ed448	114 B	57 B	Low	Listed as EdDSA	No
HMAC-SHA-256 (symmetric	32 B	—	—	Listed as a MAC	—

Scheme	Signature Size	Public Key Size	Implementation Risk	CRYPTREC	Cardano Alignment
key, reference)					
ML-DSA-44 (post-quantum, reference)	approx. 2,420 B	approx. 1,312 B	Low	Not listed	No

Bandwidth is not a deciding factor. When a block header JSON of about 1.5 KB is transmitted ten times in a carousel, the required bandwidth is about 6.3 kbps with Ed25519, about 7.0 kbps even with RSA-2048, and only about 15.7 kbps even with ML-DSA-44, all well within the 200 kbps allocated bandwidth.

A symmetric-key scheme (HMAC) is the most size-efficient, but because the verification key and the signing key are the same, anyone who reverse-engineers a single receiver could forge legitimate broadcast data. Since the main threat in this project is spoofed transmission, it cannot satisfy the requirement in principle and is excluded from consideration.

Ed25519 is adopted for three reasons. First, Cardano itself uses Ed25519 natively, and the certificate-side signatures use the same scheme, so the "two independent roots of trust" described below can be implemented on the receiver as a single verification routine. Second, because signing is deterministic, no random source is needed at signing time, which structurally eliminates private-key leakage caused by poor ECDSA nonce generation in a signer operating unattended inside the broadcast facility. Third, it is included in the CRYPTREC e-Government Recommended

Ciphers List, which justifies its use under broadcasting and public-sector procurement requirements.

Note that the Ministry of Internal Affairs and Communications (MIC) guideline "Technical Guidelines for Disaster Information Transmission Using Digital Terrestrial Broadcast Waves" requires electronic signatures (XML signatures) on EDXL but does not specify the signature algorithm or designate a certificate authority. The adoption of Ed25519 is therefore consistent with that guideline.

Two Independent Roots of Trust

Offline verification on the receiver side uses the following two sets of pre-registered information as roots of trust. Because they are independent, spoofing only one of them is not enough to pass verification.

1. The broadcast source's signature verification public key and a known block header serving as the trust anchor — used to confirm that received block headers are legitimate
2. The certificate issuer's public key — used to verify the signature of the presented certificate itself

Future Enhancements (Outside the Scope of This Project)

Using the VRF proof (block_vrf), operational certificate (op_cert), and slot leader information contained in Cardano block headers, the validity of a block could be verified at the consensus level without relying on trust in the transmission source. However, this requires the receiver to hold the stake distribution and genesis parameters, which places a heavy

implementation burden on offline receivers, so this project only records it as a future topic.

Comparison with the Existing Disaster Information Distribution System

The MIC "Technical Guidelines for Disaster Information Transmission Using Digital Terrestrial Broadcast Waves" is a national guideline that specifies almost the same transmission form as this project (distributing disaster information to outdoor loudspeaker stations and indoor receivers via IPDC on digital terrestrial broadcast waves).

Under that guideline, electronic signatures (XML signatures) must be attached to EDXL messages sent by municipalities to guarantee authenticity and prevent spoofing, and the integrated operation coordination function (back-end function) acts as a certificate authority that issues and manages the client certificates. However, the signature algorithm is not specified, and the guideline states that it "does not particularly designate the certificate authority that issues digital certificates."

The important difference for this project is where signature verification takes place. Under the guideline, electronic signatures are verified by the integrated operation coordination function and the IPDC interface equipment inside the broadcast station, and the verified data is then transmitted as broadcast waves. In other words, trust verification is completed on the sending side; the receivers (outdoor loudspeaker stations and indoor receivers) have no means of verifying received data and simply trust whatever arrives over the air. Pre-registration of public keys or root

certificates in receivers, and encryption or scrambling of broadcast data, are not specified.

The design of this project can be positioned as extending the end point of this verification all the way to the receiver. By pre-registering the signature verification public key and the trust anchor in the receiver so that the authenticity of the received data itself can be verified offline, verification is completed at the end point without relying on the sending-side facilities as a premise of trust. The fact that the guideline lists FLUTE / AL-FEC as transmission functions is also consistent with the transmission protocol stack of this design.

Key and Certificate Management and Revocation

Offline receivers cannot check revocation through CRL or OCSP. If a signing key is compromised, the broadcast waves are the only path by which receivers can learn of it. This design assumes one of the following (or both), to be finalized in M2.

- Broadcast distribution of revocation information: periodically transmit a list of revoked key IDs over IPDC and update the receiver's local cache
- Short-lived signing keys: set an expiry on signing keys and distribute rollover keys in advance over the broadcast waves (rollover keys are signed with the current key or a higher-level key)

Software Function Descriptions

Function	Description	Input	Output	Development
Block header retrieval	Retrieves block header information from Cardano	Cardano blockchain	Block header data	Update (Modification of

Function	Description	Input	Output	Development
	through an external API such as Blockfrost (reuses the existing implementation)			a previous project)
Data formatting	Formats the retrieved block header as head_{block height}.json and computes and adds block_body_hash, ipdc_hash, and ipdc_previous_block	Block header data	IPDC transmission data	Update (Modification of a previous project)
Signing	Attaches an Ed25519 (RFC 8032) digital signature to the formatted JSON and outputs it together with the signature value and key ID, enabling spoofing detection on the receiver side	IPDC transmission data	Signed IPDC transmission data	New development
FLUTE / TS generation	Compresses each bundle of signed JSON documents and its manifest, splits it for file delivery with FLUTE (RFC 3926), adds AL-FEC, and encapsulates it as UDP/IP → ULE (RFC 4326) → MPEG-2 TS	Signed IPDC transmission data	MPEG-2 TS (ASI output)	Done (existing implementation)
Transmission control	Organizes block header JSON documents into bundles and controls transmission timing and cycle according to the overlapping sliding window (bundle size N and shift S are finalized in Test 7 of B1)	IPDC transmission data	Transmission instructions to the encoder	Done (existing implementation)

Required Modifications to the Existing Implementation (Toward M2)

The implementation from the previous project (block information generation and receiver-side verification) was source-reviewed, and the items that must be modified for this demonstration are listed below.

Item	Description	Priority
Blockfrost API pagination support	The transaction list is returned at most 100 per page. The current implementation does not handle pagination and miscalculates the Merkle root for blocks with more than 100 transactions. This occurs routinely on mainnet	Required (before the experiment)
Null checks on API responses	When retrieval fails, a null reference raises an exception, and the unhandled rejection stops the generation process, leaving gaps in the chain	Required (before the experiment)
Immutability of header files	Do not overwrite generated JSON. On rollback, record the new data as a separate series and state the reconstruction in the manifest. A prerequisite for overlapping transmission	Required
Fix to rollback reconstruction	The history array allows duplicates and can reference entries from an old fork at the same height. The state is also updated before the match check, corrupting already-confirmed state	Required
Add trust anchor matching to verification	The current verification tool checks only the self-consistency of the chain and does not compare it with a block known in advance, so it accepts a self-consistently generated fake chain	Required
Add signature verification to verification	Add Ed25519 signature verification as the first stage of the verification flow	Required
Distinguish gaps from tampering	The current tool treats a jump in height as a verification failure. Compare against the list of expected heights in the manifest and report "not received" separately from	High

Item	Description	Priority
	tampering	
Operational stability	Prevent re-entry of the retrieval cycle, prevent the transmission queue file from growing unbounded, and limit the targets of old-file deletion	Medium
Decide the network to connect to	The current implementation connects to the testnet (preprod). Decide whether the experiment will run on mainnet, which will be the basis for calculating the transaction cost KPI	Required

Note that the current verification tool only checks that the chain of received block headers is mutually consistent. It does not verify that block_body_hash is the actual transaction root, nor that the chain originates from the real Cardano chain. A self-consistent fake chain generated from arbitrary initial values therefore passes the current tool. The two measures above, trust anchor matching and Ed25519 signature verification, close this gap.

Summary

This document defined the sender-side system that transmits Cardano block information over actual TV broadcast waves. The key points are as follows.

- **Architecture**: The existing block data server and our in-house IPDC system are reused. The output of the IPDC system is fed into one spare port (ASI, allocated bandwidth 200 kbps) of the broadcaster's master-control MUX and multiplexed with the regular broadcast signal for transmission.

- **Data**: Block headers are formatted as a custom JSON document (about 847 bytes per block, measured) and chained to the previous block with ipdc_hash. To give receivers more time for recovery processing, multiple blocks are grouped into bundles and transmitted redundantly with an overlapping sliding window. Even at twofold redundancy, the required bandwidth is about 0.2 kbps, roughly 0.1% of the allocated bandwidth.
- **Security**: An Ed25519 signature is attached to each block header and to each bundle's manifest. Under the existing MIC guideline, signature verification ends at the broadcaster side; this demonstration extends verification all the way to the receiver.
- **Modifications to the existing implementation**: The previous project's implementation requires nine modifications, including pagination support and the addition of trust anchor matching, which will be addressed in M2.

Remaining items, such as how the signing key is stored, how revocation information is distributed, and the bundle size and overlap width (to be determined in Test 7 of B1), will be finalized in the M2 detailed design.